

Prueba de evaluación continua de Estructuras de Datos Curso 2025-26. Grupo F

Normas de realización del examen

- Debes desarrollar e implementar una solución para el ejercicio propuesto y entregarla en el juez automático accesible en la dirección <http://exacrc>.
- Escribe tu **nombre y apellidos** en el hueco reservado para ello en los ficheros plantilla de cada ejercicio.
- Debes utilizar los ficheros plantilla de cada ejercicio, atendiendo a las instrucciones que se dan en cada fichero y escribiendo tu solución en los espacios reservados para ello, siempre entre las etiquetas `<answer>` y `<\answer>`.
- Tus soluciones serán evaluadas por la profesora independientemente del veredicto del juez automático. Para ello, la profesora tendrá en cuenta exclusivamente el último envío que hayas realizado de cada ejercicio.
- Si necesitas consultar la documentación de C++, está disponible en <http://exacrc/cppreference>

Desparizar una lista consiste en eliminar sus elementos pares (dejando únicamente los impares), añadiendo esos pares en el mismo orden al final de otra lista.

En concreto, dada la clase `ListLinkedDouble<T>` que implementa el TAD lista mediante listas doblemente enlazadas circulares con nodo fantasma y un contador con el número de elementos.

```
template <typename Elem> class ListLinkedDouble {
private:
    struct Node {
        Elem value;
        Node *next;
        Node *prev;
    };
    Node *head;
    int num_elems;
public:
    ...
    void desparizar(ListLinkedDouble & aux);
};
```

debes añadir un nuevo método `desparizar` que despariza la lista `this`, quedándose esta última con los elementos impares, y moviendo en orden los nodos con valores pares al final de la lista `aux` pasada como parámetro. Por ejemplo, dada la lista `l = [20, 4, 5, 7, 10, 3]`, y la lista `aux = [ ]`, tras la llamada

1.desparizar(aux) la lista l tiene los valores [5, 7, 3] y la lista aux los valores [20, 4, 10].

La lista aux puede no estar vacía. Si en el ejemplo anterior aux = [11, 67], después de ejecutar 1.desparizar(aux), tendremos aux = [11, 67, 20, 4, 10].

Importante: Para la implementación del método no pueden crearse, directa o indirectamente, nuevos nodos mediante new ni borrar nodos mediante delete; han de reutilizarse los nodos de la lista de entrada. Tampoco se permite copiar valores de un nodo a otro. El coste de la operación ha de ser lineal con respecto al número de elementos de this.

Entrada

La entrada comienza con un número que indica el número de casos de prueba que vienen a continuación. Cada caso de prueba consta de dos líneas. La primera describe la lista que se quiere desparizar y la segunda la lista aux. Cada lista se representa mediante una secuencia de números enteros distintos de cero, terminada en 0, que no forma parte de la lista.

Salida

Para cada caso de prueba se imprimirán dos líneas: una con el contenido de la lista original tras llamar al método desparizar, y otra con el contenido de la lista aux tras la misma llamada.

Entrada de ejemplo

```
4
1 2 3 4 5 6 7 8 9 10 0
0
2 1 0
22 0
4 6 8 0
1 0
0
2 0
```

Salida de ejemplo

```
[1, 3, 5, 7, 9]
[2, 4, 6, 8, 10]
[1]
[22, 2]
[]
[1, 4, 6, 8]
[]
[2]
```